

UNIVERSIDAD PERUANA UNIÓN

Facultad de Ingeniería y Arquitectura
Escuela Profesional de Ingeniería de Sistemas

Informe integrado de la Guía Práctica de Laboratorio 07

PharmaSoft y PharmaMobile con Ktor Client y consumo GET

Proyecto	PharmaMobile sobre PharmaSoft
Estudiante	Luciana Gutierrez
Asignatura	Desarrollo de Aplicaciones Móviles
Ciclo y semestre	VI 2026-2
Fecha de verificación	29 de septiembre de 2026

Resultado: integración GET verificada en Android con respuesta HTTP 200 y manejo recuperable del error de red.

1 Propósito y resultado

Implementé el consumo del catálogo real de PharmaSoft desde PharmaMobile mediante Ktor Client en Kotlin Multiplatform. La respuesta paginada se deserializa en DTO, se transforma al modelo de dominio y se publica a la interfaz Compose a través del repositorio, Koin, ViewModel y UiState.

La verificación del 4 de octubre de 2026 confirmó que PharmaSoft respondió correctamente, que el emulador Android mostró los tres productos creados y que la aplicación presentó un estado de error controlado al perder la conexión. La ejecución iOS no se realizó porque el entorno disponible es Windows y no se cuenta con macOS ni Xcode.

2 Entorno y alcance

Elemento	Configuración	Resultado
Backend	PharmaSoft en <code>http://localhost:8080</code>	Health y productos respondieron 200
Base de datos	Oracle FREEPDB1 en puerto 1522	Una categoría y tres productos
Android	AVD Medium Phone y 10.0.2.2:8080	Listado y error de red verificados
iOS	localhost:8080 mediante Darwin	Configurado; ejecución no realizada sin macOS

3 Implementación

3.1 Cliente HTTP multiplataforma

El catálogo de versiones declara Ktor 3.6.0, serialización Kotlin, el motor OkHttp para Android y Darwin para iOS. El HttpClient compartido se registra como una única instancia en Koin.

```
ContentNegotiation + kotlinx.serialization
Logging con LogLevel.HEADERS
HttpTimeout: 15 s de petición y 10 s de conexión
DefaultRequest con URL base y Content-Type application/json
```

3.2 Contrato, DTO y mapper

PaginaResponseDto<ProductoResponseDto> refleja el envoltorio real de PharmaSoft. ProductoResponseDto incluye id, nombre, precio, stock, estado, categoría y fechas. El mapper convierte cada DTO a Producto y mantiene el dominio libre de Ktor y de anotaciones de serialización.

```
GET productos?pagina=0&tamano=20
api.listar().contenido.map { it.toDomain() }
```

3.3 Repositorio, Koin y presentación

ProductoRepositoryImpl obtiene el listado remoto y conserva temporalmente las operaciones registrar y actualizar en el repositorio local, de acuerdo con el alcance de la sesión 7. Los errores de red se traducen a mensajes de dominio. ProductoViewModel expone UiState y Compose representa carga, contenido, vacío y error con una acción Reintentar.

4 Evaluación según los diez criterios

N.º	Criterio	Estado	Verificación
1	PharmaSoft y datos de prueba	Cumple	Backend local, una categoría y tres productos
2	Dependencias Ktor y serialización	Cumple	Catálogo y source sets configurados
3	Permisos y tráfico HTTP local	Cumple	AndroidManifest, XML de red e Info.plist
4	Motores y URL por plataforma	Cumple	OkHttp 10.0.2.2 y Darwin localhost
5	HttpClient único mediante Koin	Cumple	Registro single compartido
6	Plugins de Ktor	Cumple	Negotiation, Logging, Timeout y DefaultRequest
7	Respuesta paginada	Cumple	PaginaResponseDto y campo contenido
8	Mapper y dominio desacoplado	Cumple	toDomain sin dependencias de red en domain
9	GET 200 y lista en interfaz	Cumple	Swagger, Logcat y pantalla Android
10	Ejecución y error de red	Parcial	Android verificado; iOS no ejecutado sin macOS

5 Evidencias de ejecución

5.1 Swagger UI y productos creados

La operación GET se ejecutó con página 0, tamaño 20, ordenamiento por id y dirección ascendente. Swagger mostró HTTP 200 y el contenido persistido en PharmaSoft.

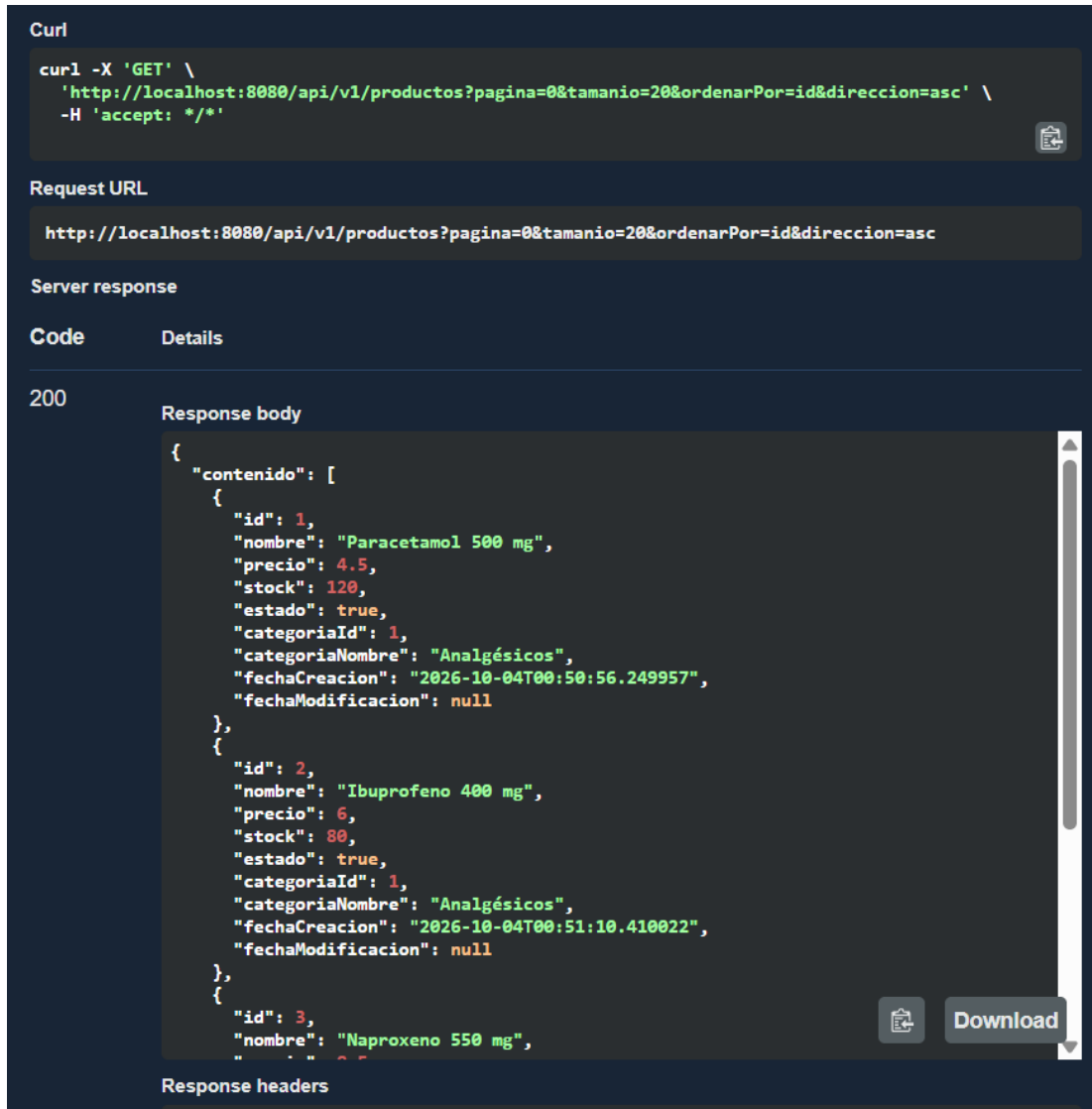


Figura 1. Swagger UI con respuesta 200 y los tres productos de PharmaSoft.

5.2 Registro de Ktor

Logcat registra la petición desde el emulador hacia 10.0.2.2 y la respuesta HTTP 200, demostrando que el cliente compartido usa la URL Android correcta.

```
Ktor / PharmaSoft - registro ADB autentico

AVD Medium_Phone | 2026-10-04 | filtro: HttpClient

10-04 06:20:27.495 I/System.out( 3478): HttpClient: REQUEST:
    http://10.0.2.2:8080/api/v1/productos?pagina=0&tamano=20

10-04 06:20:29.043 I/System.out( 3478): HttpClient: RESPONSE: 200

Fuente: logcat extraído directamente del emulador mediante ADB.
```

Figura 2. Registro de Ktor con GET al backend local y respuesta 200.

El registro original se obtuvo directamente del emulador mediante ADB. Entre la solicitud y la respuesta transcurrieron aproximadamente 1.55 segundos, dentro del tiempo máximo configurado para el cliente.

Dato observado	Valor
Host Android	10.0.2.2:8080
Endpoint	/api/v1/productos?pagina=0&tamano=20
Resultado	HTTP 200
Fuente	Logcat extraído mediante ADB

5.3 Lista renderizada en Android

La pantalla de productos muestra Paracetamol 500 mg, Ibuprofeno 400 mg y Naproxeno 550 mg con sus valores reales de precio y stock.

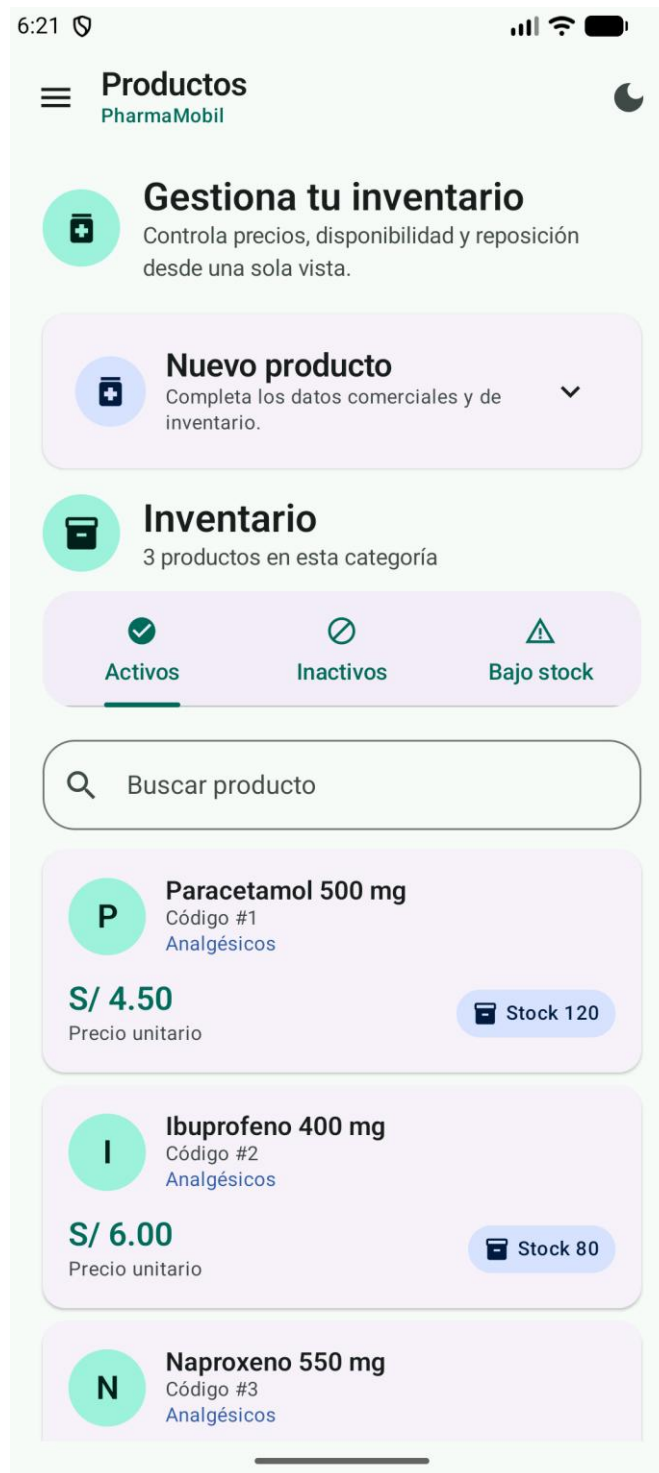


Figura 3. Catálogo de PharmaSoft renderizado en el emulador Android.

5.4 Manejo del error de conexión

Con el dispositivo sin conectividad, la aplicación no se cerró. El UiState cambió a error y Compose mostró un mensaje comprensible junto con el botón Reintentar. Tras restablecer la conexión, el listado volvió a cargarse.

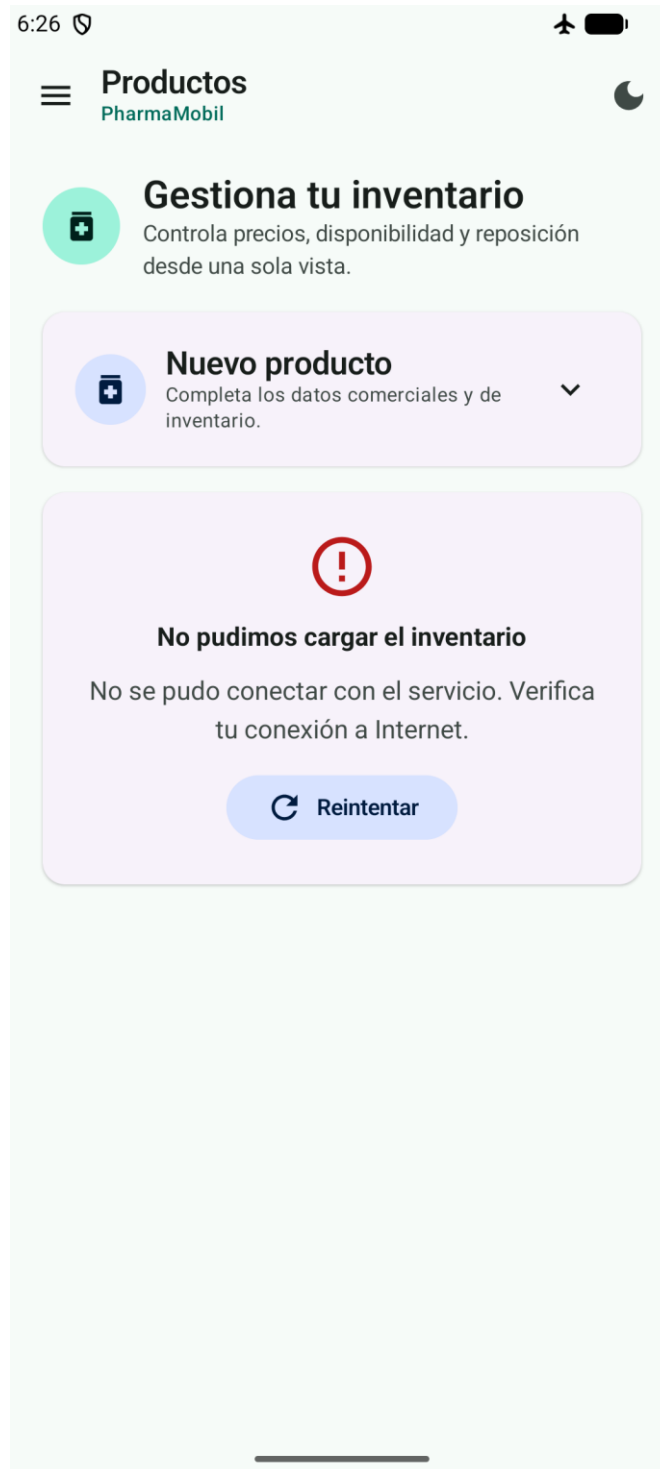


Figura 4. Estado de error recuperable mostrado en modo avión.

5.5 Verificación iOS

El código iOS incluye Darwin, la URL `http://localhost:8080/api/v1/` y `NSAllowsLocalNetworking`. No se ejecutó el simulador porque Xcode requiere macOS y el equipo disponible utiliza Windows. No se creó una evidencia sustituta ni simulada.

6 Pruebas y versionamiento

La verificación automatizada se ejecutó con el siguiente comando:

```
.\gradlew.bat :shared:testAndroidHostTest :androidApp:assembleDebug
```

Resultado: BUILD SUCCESSFUL. Se ejecutaron 18 pruebas sin fallos ni errores y se generó correctamente la APK debug.

El trabajo se entrega en la rama `feature/ktor-client-Gutierrez`. El historial conserva los commits previos de Ktor y evidencias, y añade commits coherentes para la integración final con PharmaSoft y su documentación.

7 Conclusión

La aplicación dejó de depender del listado en memoria para la consulta de productos y ahora consume el endpoint paginado de PharmaSoft. La arquitectura mantiene la separación entre red, datos, dominio y presentación; Koin administra las instancias compartidas; y la interfaz representa tanto el contenido como los fallos de conectividad sin cerrarse.

La práctica conserva una verificación pendiente: ejecutar PharmaMobile en un simulador iOS con macOS y Xcode. La evidencia Android y el manejo del error de conexión sí están disponibles. Este informe corresponde a un trabajo individual.

8 Entorno backend de PharmaSoft

Preparé el entorno local para conectar Oracle con PharmaSoft y permitir el consumo del catálogo desde PharmaMobile. Las evidencias del backend corresponden al 4 de octubre de 2026. Las tablas del proyecto fueron administradas por Flyway; no se crearon manualmente ni se modificaron las migraciones para eludir errores.

8 1 Base de datos y migraciones

Utilicé la imagen gvenzl/oracle-free:23.26.1-slim-faststart en el contenedor pharmasoft-oracle. Esta imagen proporciona Oracle AI Database 26ai Free con release 23.26.1 y admite los tipos SQL de las migraciones, incluido BOOLEAN. El puerto local 127.0.0.1:1522 se conecta con el puerto 1521 del contenedor y los datos se conservan en el volumen pharmasoft-oracle-data.

El servicio FREEPDB1 quedó abierto en modo READ WRITE. El esquema PHARMADB utiliza la credencial local configurada por el proyecto, con CREATE SESSION, CREATE TABLE y CREATE SEQUENCE y una cuota de 100 MB en USERS. La contraseña no se reproduce en este informe y no debe reutilizarse en producción.

La conexión JDBC es jdbc:oracle:thin:@//localhost:1522/FREEPDB1. Flyway aplicó correctamente V1__esquema_inicial.sql y registró su ejecución en la tabla de historial. Se verificaron cinco tablas del dominio y la tabla de historial de Flyway.

8 2 Ejecución y comprobaciones

PharmaSoft se ejecutó con Java 21.0.12 LTS, el wrapper Maven del proyecto y el perfil dev. La compilación y las pruebas del backend finalizaron correctamente con 21 pruebas, sin fallos ni errores. La aplicación atendió en el puerto 8080.

GET <http://localhost:8080/api/health> respondió HTTP 200 y el texto Backend funcionando correctamente. Swagger estuvo disponible en <http://localhost:8080/swagger-ui.html> y la descripción OpenAPI en [/v3/api-docs](#).

Estos resultados describen las verificaciones realizadas durante la práctica, no garantizan que los servicios permanezcan encendidos después de cerrar el equipo. Para repetir la prueba se debe iniciar Docker y el contenedor Oracle, esperar su disponibilidad y ejecutar `.\mvnw.cmd spring-boot:run` con Java 21.

9 Datos y evidencias del backend

La categoría Analgésicos con ID 1 ya existía. Un intento repetido de alta devolvió 409 por nombre duplicado; se reutilizó la categoría sin borrarla ni recrearla. La figura 5 acredita su existencia con GET 200, no el POST original de la categoría.

Creé desde Swagger los productos Paracetamol 500 mg con precio 4.50 y stock 120, Ibuprofeno 400 mg con precio 6.00 y stock 80 y Naproxeno 550 mg con precio 8.50 y stock 60. Los tres utilizan categoriaId 1 y cada POST respondió 201 Created.

Swagger muestra 201 como Undocumented porque la descripción de esa operación anuncia 200. El código HTTP real 201 indica creación exitosa. No se cambió la lógica funcional para eliminar esa etiqueta.

GET /api/v1/productos?pagina=0&tamano=20&ordenarPor=id&direccion=asc respondió 200. Su objeto paginado contiene los tres productos, totalElementos 3, totalPaginas 1 y ultima true. El cuerpo completo está en evidencias/sesion07/productos-get.json del repositorio backend.

9 1 Repositorios y procedencia

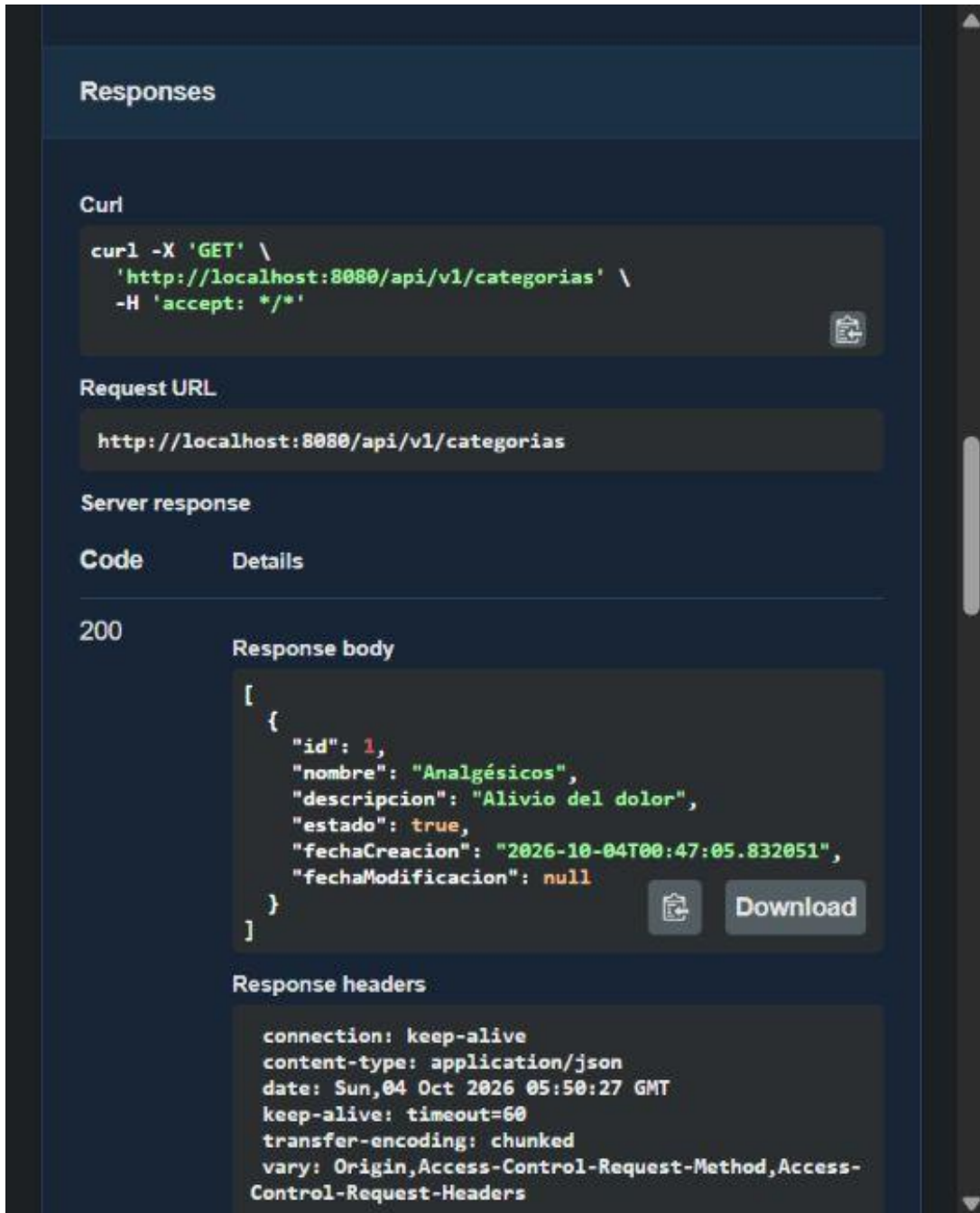
Backend publicado en <https://github.com/LucianaGutierrez-049/Backend>, rama main. Los commits 8c025b1, a808459 y 91e9b96 corresponden a la importación con atribución, la documentación del entorno y las evidencias reales de Swagger. PharmaSoft parte del proyecto <https://github.com/dreyna/pharmaSoft>; PROCEDENCIA.md conserva la atribución de esa base. Los commits nuevos no implican autoría del código original importado.

PharmaMobile se entrega en <https://github.com/LucianaGutierrez-049/PharmaMobile/tree/feature/ktor-client-Gutierrez>. El reporte móvil registra los commits 9975f5a, dbbcb55, 9afdba3 y bd0ceaf y 18 pruebas exitosas. El resultado móvil descrito en las secciones anteriores no se ejecutó nuevamente al integrar este documento.

9 2 Pendientes de entrega

Falta la ejecución real y captura iOS en macOS con Xcode, o la confirmación del docente de que acepta esa limitación. No se dispone de captura del POST original de la categoría. La entrega en el aula virtual y la aceptación de la modalidad individual deben confirmarse por separado. El POST desde PharmaMobile queda fuera del alcance de la sesión 7.

Categoría existente



The screenshot displays a REST client interface with a dark theme. The 'Responses' tab is active, showing the details of a successful GET request. The 'Curl' section contains the command: `curl -X 'GET' \ 'http://localhost:8080/api/v1/categorias' \ -H 'accept: */*'` . The 'Request URL' is `http://localhost:8080/api/v1/categorias` . The 'Server response' section shows a status code of 200. The 'Response body' is a JSON array containing one object for the 'Analgésicos' category. The 'Response headers' section lists various headers including 'connection: keep-alive', 'content-type: application/json', and 'date: Sun, 04 Oct 2026 05:50:27 GMT'.

Responses

Curl

```
curl -X 'GET' \
'http://localhost:8080/api/v1/categorias' \
-H 'accept: */*' 
```

Request URL

```
http://localhost:8080/api/v1/categorias 
```

Server response

Code	Details
200	Response body <pre>[{ "id": 1, "nombre": "Analgésicos", "descripcion": "Alivio del dolor", "estado": true, "fechaCreacion": "2026-10-04T00:47:05.832051", "fechaModificacion": null }]</pre> Response headers <pre>connection: keep-alive content-type: application/json date: Sun, 04 Oct 2026 05:50:27 GMT keep-alive: timeout=60 transfer-encoding: chunked vary: Origin, Access-Control-Request-Method, Access-Control-Request-Headers</pre>

Figura 5. GET de la categoría Analgésicos con respuesta 200.

Alta de Paracetamol

The screenshot displays a REST client interface with the following sections:

- Curl:** A code block containing the curl command for a POST request to `http://localhost:8080/api/v1/productos` with headers `accept: */*` and `Content-Type: application/json`. The body is a JSON object: `{ "nombre": "Paracetamol 500 mg", "precio": 4.5, "stock": 120, "estado": true, "categoriaId": 1 }`.
- Request URL:** A text field containing `http://localhost:8080/api/v1/productos`.
- Server response:** A section showing the response details.
 - Code:** 201, labeled as *Undocumented*.
 - Response body:** A JSON object: `{ "id": 1, "nombre": "Paracetamol 500 mg", "precio": 4.5, "stock": 120, "estado": true, "categoriaId": 1, "categoriaNombre": "Analgésicos", "fechaCreacion": "2026-10-04T00:50:56.2499577", "fechaModificacion": null }`. A **Download** button is present next to the body.
 - Response headers:** A list of headers: `connection: keep-alive`, `content-type: application/json`, `date: Sun, 04 Oct 2026 05:50:56 GMT`, `keep-alive: timeout=60`, `transfer-encoding: chunked`, and `vary: Origin, Access-Control-Request-Method, Access-`.

Figura 6. POST de Paracetamol 500 mg con respuesta 201.

Alta de Ibuprofeno

The screenshot displays a REST client interface with the following sections:

- Curl:** A code block containing the curl command for a POST request to `http://localhost:8080/api/v1/productos` with headers `accept: */*` and `Content-Type: application/json`. The body is a JSON object: `{ "nombre": "Ibuprofeno 400 mg", "precio": 6, "stock": 80, "estado": true, "categoriaId": 1 }`.
- Request URL:** A text field showing `http://localhost:8080/api/v1/productos`.
- Server response:** A section with a table header showing a **201** status code, labeled as *Undocumented*. The **Response body** is a JSON object: `{ "id": 2, "nombre": "Ibuprofeno 400 mg", "precio": 6, "stock": 80, "estado": true, "categoriaId": 1, "categoriaNombre": "Analgésicos", "fechaCreacion": "2026-10-04T00:51:10.4100224", "fechaModificacion": null }`. A **Download** button is present next to the response body.
- Response headers:** A text area showing headers: `connection: keep-alive`, `content-type: application/json`, `date: Sun, 04 Oct 2026 05:51:10 GMT`, `keep-alive: timeout=60`, `transfer-encoding: chunked`, and `vary: Origin, Access-Control-Request-Method, Access-`.

Figura 7. POST de Ibuprofeno 400 mg con respuesta 201.

Alta de Naproxeno

The screenshot displays a REST client interface with the following sections:

- Curl:** A code block containing the curl command for a POST request to `http://localhost:8080/api/v1/productos` with headers `accept: */*` and `Content-Type: application/json`. The body is a JSON object: `{ "nombre": "Naproxeno 550 mg", "precio": 8.5, "stock": 60, "estado": true, "categoriaId": 1 }`.
- Request URL:** A text field containing `http://localhost:8080/api/v1/productos`.
- Server response:** A section showing the response details.
 - Code:** 201, labeled as *Undocumented*.
 - Response body:** A JSON object: `{ "id": 3, "nombre": "Naproxeno 550 mg", "precio": 8.5, "stock": 60, "estado": true, "categoriaId": 1, "categoriaNombre": "Analgésicos", "fechaCreacion": "2026-10-04T00:51:24.9347381", "fechaModificacion": null }`. A **Download** button is present next to the body.
 - Response headers:** A list of headers: `connection: keep-alive`, `content-type: application/json`, `date: Sun, 04 Oct 2026 05:51:24 GMT`, `keep-alive: timeout=60`, `transfer-encoding: chunked`, and `vary: Origin, Access-Control-Request-Method, Access-`.

Figura 8. POST de Naproxeno 550 mg con respuesta 201.

Consulta de productos

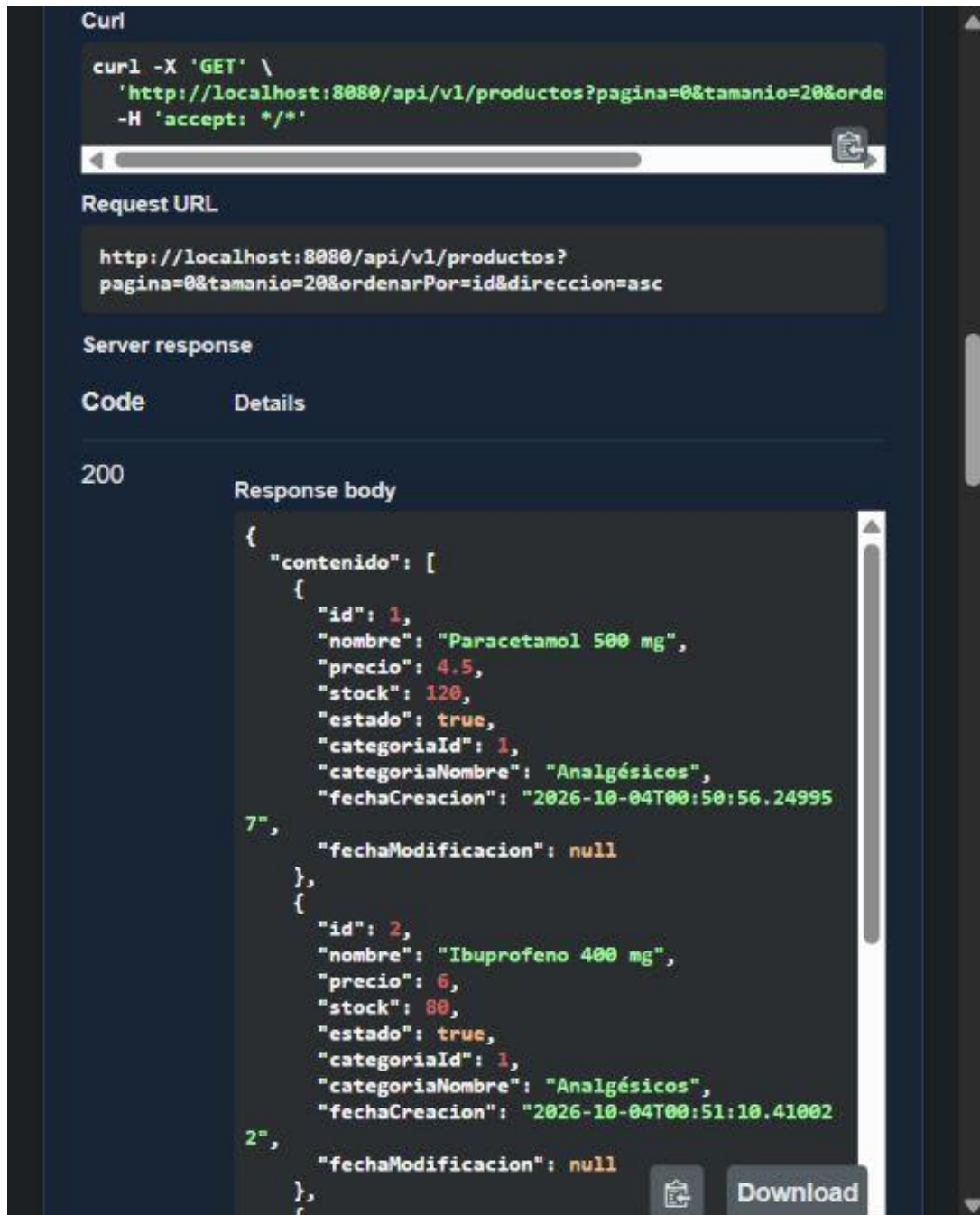
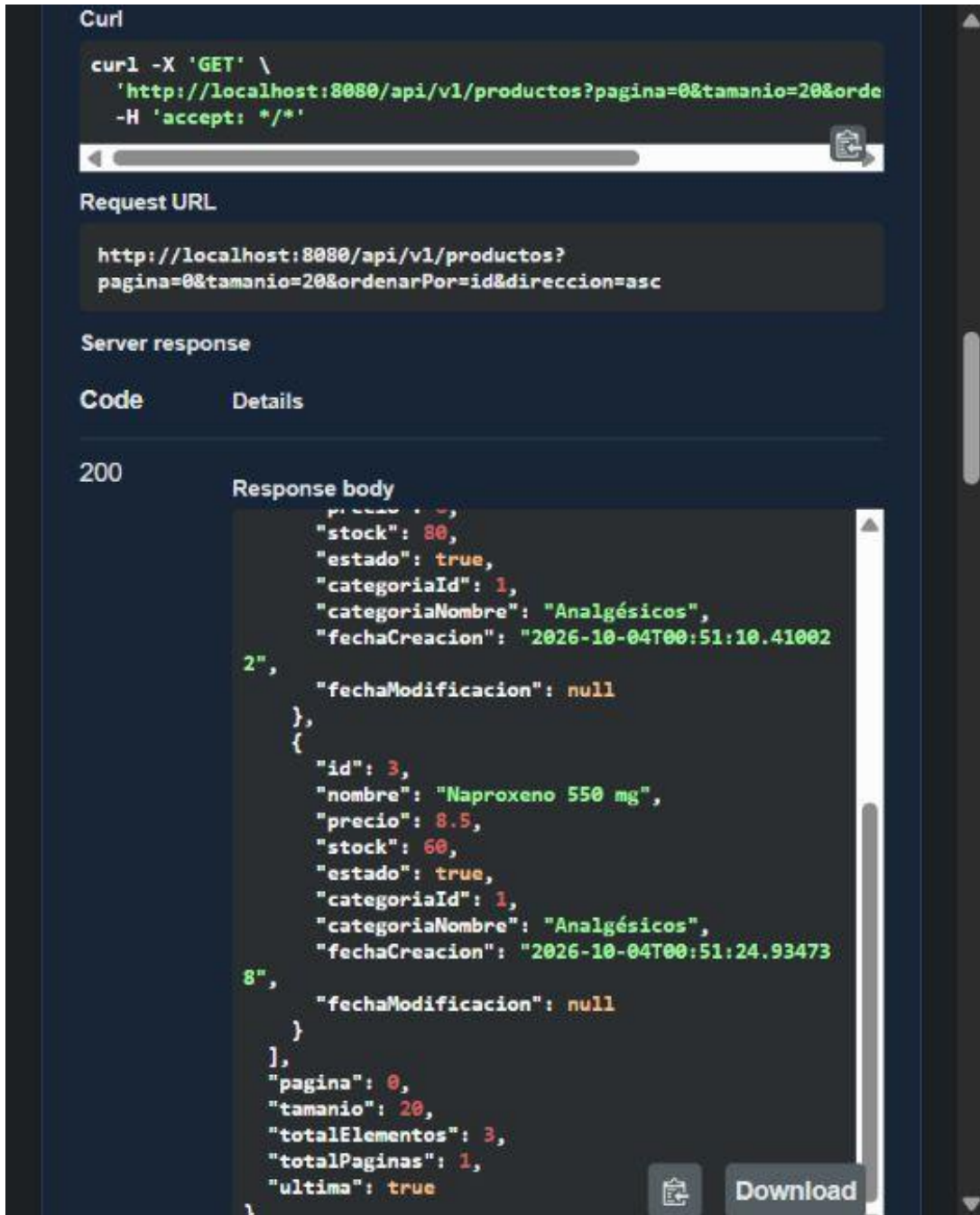


Figura 9. GET 200 del catálogo con el inicio de la respuesta paginada.

Paginación de productos



The screenshot shows a REST client interface with the following sections:

- Curl:**

```
curl -X 'GET' \
  'http://localhost:8080/api/v1/productos?pagina=0&tamano=20&ordenarPor=id&direccion=asc'
-H 'accept: */*'
```
- Request URL:** `http://localhost:8080/api/v1/productos?pagina=0&tamano=20&ordenarPor=id&direccion=asc`
- Server response:** **Code** 200, **Details**
- Response body:** A JSON array of 3 products and pagination metadata.

```
{
  "productos": [
    {
      "id": 1,
      "nombre": "Paracetamol 500 mg",
      "precio": 1.2,
      "stock": 80,
      "estado": true,
      "categoriaId": 1,
      "categoriaNombre": "Analgésicos",
      "fechaCreacion": "2026-10-04T00:51:10.41002",
      "fechaModificacion": null
    },
    {
      "id": 2,
      "nombre": "Ibuprofeno 400 mg",
      "precio": 2.5,
      "stock": 50,
      "estado": true,
      "categoriaId": 1,
      "categoriaNombre": "Analgésicos",
      "fechaCreacion": "2026-10-04T00:51:10.41002",
      "fechaModificacion": null
    },
    {
      "id": 3,
      "nombre": "Naproxeno 550 mg",
      "precio": 8.5,
      "stock": 60,
      "estado": true,
      "categoriaId": 1,
      "categoriaNombre": "Analgésicos",
      "fechaCreacion": "2026-10-04T00:51:24.93473",
      "fechaModificacion": null
    }
  ],
  "pagina": 0,
  "tamano": 20,
  "totalElementos": 3,
  "totalPaginas": 1,
  "ultima": true
}
```

Figura 10. Continuación del GET con totalElementos 3 y totalPaginas 1.

Estado del backend

Responses

Curl

```
curl -X 'GET' \
  'http://localhost:8080/api/health' \
  -H 'accept: */*'
```

Request URL

```
http://localhost:8080/api/health
```

Server response

Code	Details
200	Response body <pre>Backend funcionando correctamente</pre> Download Response headers <pre>connection: keep-alive content-length: 33 content-type: text/plain;charset=UTF-8 date: Sun, 04 Oct 2026 05:52:42 GMT keep-alive: timeout=60 vary: Origin, Access-Control-Request-Method, Access-Control-Request-Headers</pre>

Responses

Code	Description	Links
------	-------------	-------

Figura 11. GET de api health con respuesta 200.